Εργαστήριο 1:
SPIM: A MIPS32 Simulator

ΕΕΙ 233 (Lab): ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΜΙΧΑΛΗΣ ΠΙΓΓΟΣ

michalis.pingos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Outline

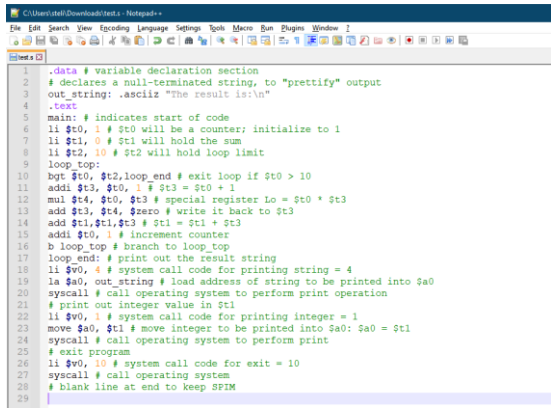
- Εγκατάσταση επεξεργαστή κειμένου
- Εγκατάσταση εξομοιωτή MIPS – QTspim
- Κατανόηση δομής προγραμμάτων σε MIPS
- Πρακτική στον εξομοιωτή QTspim
- Παραδείγματα

2

Εξομοιωτής QTSpim & κειμενογράφος notepad++

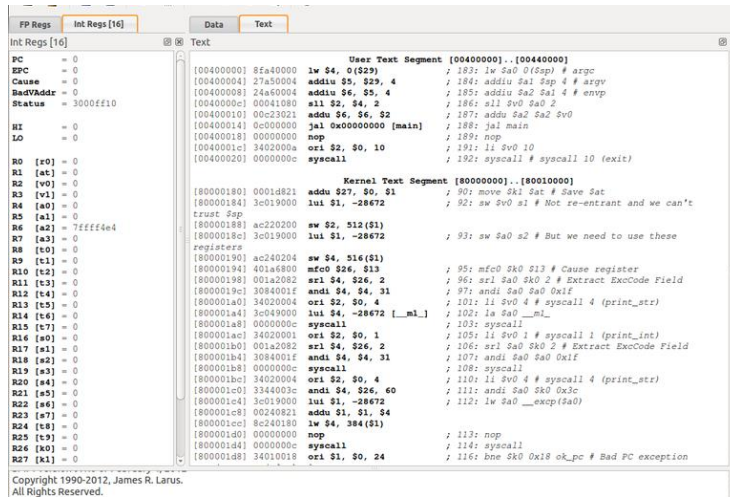
- Sample Text Editor:
  - https://notepad-plus-plus.org/downloads/
- SPIM: A MIPS32 Simulator:
  - https://spimsimulator.sourceforge.net/
  - Download link:
  - https://sourceforge.net/projects/spimsimulator/files/

Notepad ++



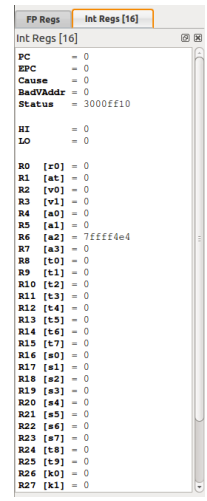
- Τα προγράμματα πρέπει να αποθηκεύονται με extension *.s
- Μην αχρησιμοποίητε ελληνικούς χαρακτήρες στο όνομα ή στο path του αρχείου σας

QTSpim



Register Panel

- Οι καταχωρητές διαχωρίζονται σε:
  - Γενικοί καταχωρητές: R0 – R31
  - Ειδικοί Καταχωρητές:
    - Program Counter
    - Etc.

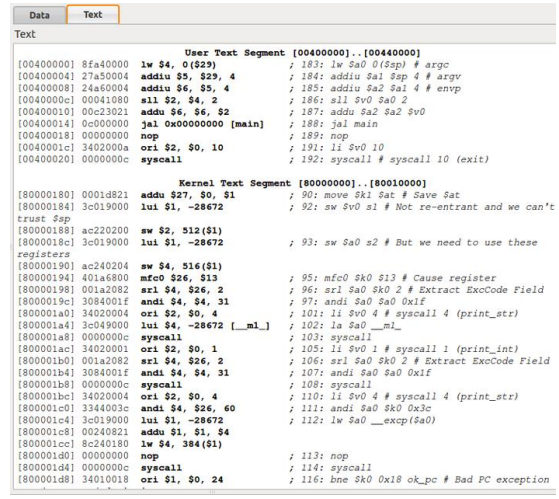


Memory Panel

Διαχωρίζεται σε 2 παράθυρα: Data και Text

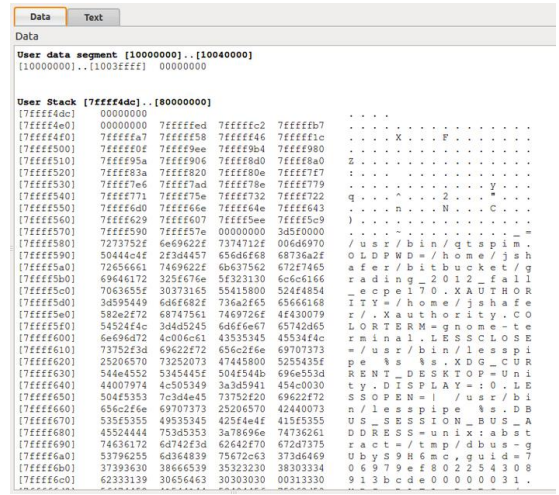
Παράθυρο Text

- Η διεύθυνση μνήμης μιας εντολής σε δεκαεξαδικό (εμφανίζεται σε αγκύλες)
- Οι πραγματικές εντολές που εκτελεί ο επεξεργαστής (σε δυαδικό)
- Η κώδικας assembly χρησιμοποιώντας τα ονόματα των καταχωρητών
- Το πρόγραμμα MIPS που εκτελείται



Παράθυρο Data

- Η καρτέλα δεδομένων εμφανίζει τα περιεχόμενα της μνήμης
- Αυτό περιλαμβάνει τις μεταβλητές και τους πίνακες που χρησιμοποιούνται, συμπεριλαμβανομένου και το περιεχόμενο της στοίβας



Messages Panel

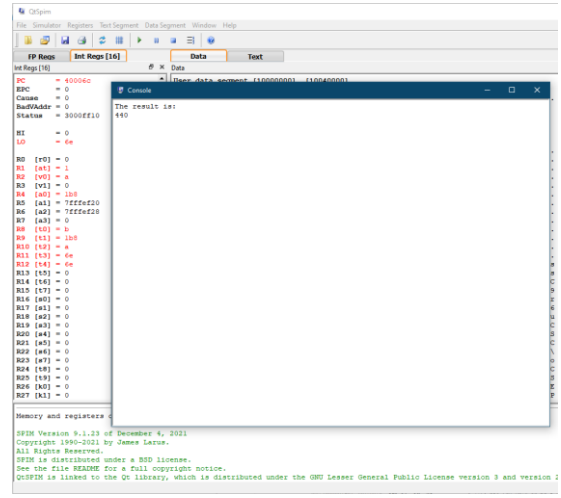
- Εμφανίζονται τα μηνύματα προς τον προγραμματιστή από τον εξομοιωτή

```
Memory and registers cleared

SPIM Version 9.1.23 of December 4, 2021
Copyright 1990-2021 by James Larus.
All Rights Reserved.
SPIM is distributed under a BSD license.
See the file README for a full copyright notice.
QtSPIM is linked to the Qt library, which is distributed under the GNU Lesser General Public License version 3 and version 2.1.
```

Console

- Στο Console (2ο παράθυρο) εμφανίζεται το αποτέλεσμα του προγράμματος



Γράφοντας προγράμματα MIPS σε SPIM

- Οι ορισμοί δεδομένων αρχίζουν με ρυθμιστές .data
- Ορισμοί κώδικα αρχίζουν με τον ρυθμιστή .text
  - Ο όρος ".text" είναι το παραδοσιακό όνομα για τη μνήμη που κρατά ένα πρόγραμμα.
- **Labels:** Σε γενικές γραμμές υπάρχουν διάφοροι ορισμοί υπό-ρουτινών και πάντα ένα "main".

```
.data ## Data declaration section

out_string: .asciiz "\nHello, World!\n"

.text ## Assembly language instructions

main: ## Start of code section
    li $v0, 4
    la $a0, out_string
    syscall

    li $v0, 10 # terminate program
    syscall
```

Γράφοντας προγράμματα MIPS σε SPIM

- Μπορείτε επίσης να καθορίσετε μεταβλητές και σταθερές με:
  - .word: καθορίζει ποσότητες 32 bits
  - .byte: καθορίζει ποσότητες 8 bits
  - .asciiz: γραμματοσειρές σε κωδικοποίηση ASCII
  - .space: αναθέτει μερικά bytes

- Μερικά παραδείγματα:

```
.data
   prompt:   .asciiz "Hello World\n"
   msg: .asciiz "The answer is "
   x:    .space 4
   y:    .word 4
   str: .space 100
```


Απλές εντολές εισόδου/εξόδου

- Ο SPIM παρέχει μερικές απλές εντολές εισόδου/εξόδου με την εντολή "syscall" (system call). Η συγκεκριμένη I/O εξαρτάται από μερικούς καταχωρητές και συγκεκριμένα:
  - Ο **$v0** για να υποδείξει τη λειτουργία
  - Μια παραμέτρους στους καταχωρητές **$a0** ή **$a1**

- Ένα κάλεσμα προς το σύστημα (system call) χρησιμοποιείται για την επικοινωνία με το σύστημα στην εκτέλεση εντολών εισόδου/εξόδου
- Φορτώστε ένα κάλεσμα προς το σύστημα διαμέσου του καταχωρητή **$v0**
- Φορτώστε επιχειρήματα (arguments), εάν υπάρχουν, στους καταχωρητές **$a0** και **$a1**
- Καλώντας ένα syscall, τα αποτελέσματα επιστρέφεται στον καταχωρητή **$v0**

MIPS System Calls

- Περισσότερα στο Appendix A του βιβλίου

Table: System services.

Service	System Call Code	Arguments	Result
print_int	1	$a0 = integer	
print_float	2	$f12 = float	
print_double	3	$f12 = double	
print_string	4	$a0 = string	
read_int	5		integer (in $v0)
read_float	6		float (in $f0)
read_double	7		double (in $f0)
read_string	8	$a0 = buffer, $a1 = length	
sbrk	9	$a0 = amount	address (in $v0)
exit	10		
print_character	11	$a0 = character	
read_character	12		character (in $v0)
open	13	$a0 = filename,	file descriptor (in $v0)
		$a1 = flags, $a2 = mode	
read	14	$a0 = file descriptor,	bytes read (in $v0)
		$a1 = buffer, $a2 = count	
write	15	$a0 = file descriptor,	bytes written (in $v0)
		$a1 = buffer, $a2 = count	
close	16	$a0 = file descriptor	0 (in $v0)
exit2	17	$a0 = value	

Παράδειγμα 1 – Hello World

```
# Hello, World!

.data ## Data declaration section

## String to be printed:
out_string: .asciiz "\nHello, World!\n"

.text ## Assembly language instructions go in text segment

main: ## Start of code section
    li $v0, 4 # system call code for printing string = 4
    la $a0, out_string # load address of string to be printed into $a0
    syscall # call operating system to perform operation
            # specified in $v0
            # syscall takes its arguments from $a0, $a1, ...
    li $v0, 10 # terminate program
    syscall
```

Παράδειγμα 2 – Αριθμητικές Εντολές

```
.data
# Data section to store variables
num1: .word 10
num2: .word 20
result: .word 0
newline: .asciiz "\n"

.text
# Code section
.globl main
main:
    # Load first number into register $t0
    lw $t0, num1
    # Load second number into register $t1
    lw $t1, num2
    # Add two numbers
    add $t2, $t0, $t1
    # Store result in register $t2 into memory
    sw $t2, result
    # Load result into $a0 to display it
    lw $a0, result
    # System call to print integer from $a0
    li $v0, 1
    syscall
    # System call to print a new line
    li $v0, 4
    la $a0, newline
    syscall
    # System call to exit program
    li $v0, 10
    syscall
```


17

Παράδειγμα 3 – Εντολές Μνήμης

```
.data
# Data section to store variables
num: .word 42
result: .word 0

.text
# Code section
.globl main
main:
    # Load the value stored in memory location 'num' into register $t0
    lw $t0, num

    # Store the value in register $t0 into memory location 'result'
    sw $t0, result

    # Load the value stored in memory location 'result' into register $a0
    lw $a0, result

    # System call to print integer
    li $v0, 1
    syscall

    # System call to exit program
    li $v0, 10
    syscall
```


18

Παράδειγμα 4 – Branches & Jumps

```
.data
# Data section to store variables
num1: .word 10
num2: .word 20
result: .word 0

.text
# Code section
.globl main
main:
    lw $t0, num1 # Load first number into register $t0
    lw $t1, num2  # Load second number into register $t1
    # Check if first number is greater than second number
    bgt $t0, $t1, greater
    # If first number is not greater than second number, jump to 'not_greater' label
    j not_greater

# Label for if first number is greater than second number
greater:
    # Store result as 1 in memory location 'result'
    li $t2, 1
    sw $t2, result
    j end  # Jump to end of program
# Label for if first number is not greater than second number
not_greater:
    # Store result as 0 in memory location 'result'
    li $t2, 0
    sw $t2, result
# Label to end of program
end:
    lw $a0, result # Load result into $a0 to display it
    li $v0, 1  # System call to print integer
    syscall
    # System call to exit program
    li $v0, 10
    syscall
```


Εργαστήριο 1:
SPIM: A MIPS32 Simulator

ΕΕΙ 233 (Lab): ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΜΙΧΑΛΗΣ ΠΙΓΓΟΣ

michalis.pingos@cut.ac.cy